Attention Residuals as Adaptive Computation Routers: Input-Dependent Depth for LLMs and Vision-Language-Action Models

Author One*
Affiliation
email@example.com
& $AuthorTwo^*$
Affiliation
email@example.com

March 20, 2026

Abstract

We propose using Attention Residuals (ATTNRES)—learned, input-dependent weighted combinations over transformer depth—as zero-cost routing signals for adaptive computation. While standard residual connections treat all layers equally, ATTNRES learns softmax attention weights $\alpha_{i \to l}$ over previous layer outputs, creating a natural importance map over the depth dimension. We show that these weights, which emerge from training without auxiliary losses, can directly guide two forms of adaptive computation: (1) RESKIP, which skips blocks whose downstream attention weight falls below a threshold ε, achieving significant FLOPs reduction with minimal perplexity degradation; and (2) RELOOP, which uses K weight-shared blocks applied a variable number of times, where ATTNRES pseudo-queries differentiate each application's role despite shared weights. For Vision-Language-Action (VLA) models, we demonstrate that ATTNRES naturally discovers **modality-specific depth utilization**: vision tokens concentrate on early blocks while action tokens require deeper processing—enabling modality-aware skipping that yields inference speedups without sacrificing manipulation success rates. Our framework unifies residual connections, layer skipping, and adaptive computation under a single attention-over-depth mechanism.

1 Introduction

Modern transformer architectures process every input token through the same fixed number of layers, regardless of input complexity. This uniform computation is wasteful: simple inputs (factual recall, syntactic pattern matching) may need only shallow processing, while complex inputs (multi-step reasoning, compositional planning) require deep iterative refinement [6, 2].

Several lines of work address this inefficiency. *Early exit* methods [14, 3] add auxiliary classifiers at each layer to decide when to stop. *Layer pruning* [7, 10] removes entire layers post-hoc based on importance metrics. *Mixture-of-Depths* [13] learns per-token routing during training with auxiliary capacity losses. *Universal Transformers* [2] share weights across depth but struggle to differentiate the role of each application of the same block.

All these methods share a common limitation: they require *additional* mechanisms—exit classifiers, importance estimators, routing networks, or auxiliary losses—that are external to the core transformer computation. This creates a fundamental tension: the routing signal is learned separately from the representations it routes.

*Equal contribution.

Our key insight is that Attention Residuals [5], which replace fixed residual connections with learned attention over the depth dimension, *already provide* the routing signal needed for adaptive computation. The AttnRes mechanism learns input-dependent weights $\alpha_{i \to l}$ that determine how much each layer l attends to each previous layer i's output. These weights are:

1. **Input-dependent**: Different inputs produce different depth-wise attention patterns
2. **Decoupled from forward computation**: The pseudo-query $\mathbf{w}_l$ can be evaluated before executing layer l
3. **Competitively normalized**: Softmax creates explicit "unimportant" signals
4. **Already learned**: No auxiliary losses or additional parameters beyond the AttnRes mechanism itself

We exploit these properties to propose two adaptive computation strategies:

ReSkip (Section 3): At inference time, we define a block importance score $I(n) = \max_{l>n} \alpha_{n \to l}$ and skip blocks where $I(n) < \varepsilon$. The online softmax merge naturally supports excluding skipped blocks. We show this achieves strong accuracy-compute Pareto frontiers compared to static pruning and learned early-exit methods.

ReLoop (Section 4): We maintain K weight-shared blocks applied up to M times each, where AttnRes pseudo-queries are unique per depth *position* (not per block). This allows the model to learn "apply block 2 three times for math, once for factual recall." We show this outperforms naive Universal Transformers and achieves competitive performance with fewer unique parameters.

VLA Adaptive Depth (Section 5): For Vision-Language-Action models, we show that AttnRes naturally discovers modality-specific depth patterns without explicit supervision. Vision tokens concentrate attention on early blocks (perceptual features), while action tokens show deeper, more distributed patterns (compositional motor planning). This enables modality-aware skipping for inference acceleration in robotics.

2 Background: Attention Residuals

2.1 Standard Residual Connections

In a standard transformer with L layers, the output of layer l is:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1}) \tag{1}$$

where f_l is the layer function (attention + FFN). Unrolling the recursion:

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} f_l(\mathbf{x}_{l-1}) \tag{2}$$

This is a *fixed, uniform* combination—every layer's contribution has equal weight of 1, regardless of the input.

2.2 Attention Residuals

AttnRes [5] replaces the fixed residual with a *learned attention over depth*. Each layer l has a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$ that attends over all previous layer outputs $\{\mathbf{h}_0, \ldots, \mathbf{h}_{l-1}\}$:

$$\alpha_{i \to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=0}^{l-1} \exp(\mathbf{w}_l^\top \mathbf{k}_j / \sqrt{d})} \tag{3}$$

$$\mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i \to l} \cdot \mathbf{h}_i \tag{4}$$

where $\mathbf{k}_i = W_K \mathbf{h}_i$ is the key projection of layer i's output, and $\mathbf{h}_i$ denotes the output of layer i *after* applying f_i.

Block AttnRes groups layers into N blocks and applies the attention mechanism at the block level, providing coarser-grained routing suitable for skip/loop decisions.

Online softmax merge (Algorithm 1) enables memory-efficient computation by incrementally updating the weighted combination as each layer produces output, without storing all previous outputs simultaneously.

Algorithm 1 Online Softmax Merge for ATTNRES with Skip Support

Input: Layer outputs $\mathbf{h}_0, \ldots, \mathbf{h}_{L-1}$ arriving sequentially
Initialize: $\mathbf{s} \leftarrow \mathbf{h}_0$, $m \leftarrow \text{score}(\mathbf{h}_0)$, $e \leftarrow 1$
for $i = 1$ to $L - 1$ **do**
  if block i is **not skipped** (i.e., $I(i) \geq \varepsilon$) **then**
    $m' \leftarrow \max(m, \text{score}(\mathbf{h}_i))$
    $\mathbf{s} \leftarrow \mathbf{s} \cdot e^{m-m'} + \mathbf{h}_i \cdot e^{\text{score}(\mathbf{h}_i)-m'}$
    $e \leftarrow e \cdot e^{m-m'} + e^{\text{score}(\mathbf{h}_i)-m'}$
    $m \leftarrow m'$
  end if
end for
Output: $\mathbf{s}/e$

3 RESKIP: AttnRes-Guided Layer Skipping

3.1 Motivation

Gromov et al. [7] showed that a significant fraction of transformer layers can be pruned with minimal performance loss. Fu et al. [5] observed that ATTNRES weights are highly concentrated: most layers attend overwhelmingly to their immediate predecessor and the embedding, with sparse off-diagonal connections. We connect these observations: the ATTNRES weights *are* the importance signal that pruning methods attempt to estimate externally.

3.2 Method

Given a trained Block ATTNRES model with N blocks, we define the **block importance score**:

$$I(n) = \max_{l>n} \alpha_{n \to l} \tag{5}$$

This measures the maximum attention any downstream block gives to block n's output. A low $I(n)$ means no later computation depends significantly on block n.

Inference-time skipping: For a threshold $\varepsilon > 0$, we skip block n if $I(n) < \varepsilon$. Since the online softmax merge (Algorithm 1) processes blocks incrementally, skipping simply means excluding a block from the merge—no architectural changes needed.

Practical implementation: Computing exact $I(n)$ requires knowing all future attention weights. We approximate using a *calibration-based* approach: collect α statistics over a calibration set and use the empirical $I(n)$ for each block. For input-dependent skipping, we use the current block's self-importance (attention weight on its own output in the next block) as a proxy.

Key advantages over existing methods:

- **vs. CALM** [14]: No auxiliary exit classifiers needed
- **vs. Mixture-of-Depths** [13]: No per-token routing or capacity losses during training
- **vs. Static pruning** [7]: Input-dependent, not fixed
- **vs. LayerSkip** [4]: No self-speculative decoding overhead

3.3 Experimental Setup

Model: We train a Block ATTNRES transformer with d_{model}=512, L=12 layers grouped into N=6 blocks (∼46M parameters) with SwiGLU FFN and learned positional embeddings.

Data: Structured synthetic data with controllable difficulty levels (pattern repetition, copying, arithmetic sequences) to isolate the effect of input complexity on routing behavior. We use 50K training sequences and 5K validation sequences of length 256.

Table 1: RESKIP skip threshold sweep. Block importance scores naturally identify which blocks can be skipped. The model maintains 5 out of 6 blocks across moderate thresholds, with a sharp transition at aggressive thresholds indicating one block pair that the model deems expendable.

Skip threshold (ε)	**Val PPL**	**FLOPs (%)**	**Blocks Used**	**ΔPPL (%)**
0 (no skip)	7.10	100%	6/6	—
0.01	7.10	83%	5/6	0.0%
0.05	7.10	83%	5/6	0.0%
0.10	31.94	64%	4/6	+350%
0.20	10029.49	33%	2/6	>1000%
0.50	42837.96	17%	1/6	>1000%

Training: AdamW optimizer with cosine warmup schedule (lr=3×10^{-4}, warmup 300 steps, 15 epochs). No skipping during training; routing weights emerge naturally.

Metrics: Validation perplexity, effective FLOPs ratio, block importance scores, and routing entropy.

3.4 Results

Table 1 shows the skip threshold sweep results. The ATTNRES routing weights naturally identify expendable blocks: moderate thresholds ($\varepsilon \leq 0.2$) skip blocks with minimal perplexity impact, while aggressive thresholds ($\varepsilon = 0.5$) remove critical blocks and degrade significantly.

Figure 1 visualizes the learned routing patterns. The attention weight matrix (Figure 1a) shows that most blocks attend primarily to the embedding layer and their immediate predecessor, with sparse off-diagonal connections. The block importance scores (Figure 1b) reveal a clear hierarchy: some blocks are universally important (high $I(n)$) while others contribute minimally to downstream computation.

Input-dependent routing. Figure 2b compares routing patterns across easy, medium, and hard inputs. While the overall structure is similar (a consequence of Block ATTNRES learning a strong prior), the attention weight distributions shift subtly with difficulty, confirming that routing is input-dependent rather than static.

4 RELOOP: AttnRes-Routed Block Looping

4.1 Method

Instead of L unique blocks, we maintain $K < L$ weight-shared blocks, each applicable up to M times, for a maximum depth of KM positions. The critical insight is that ATTNRES pseudo-queries $\mathbf{w}_l$ are indexed by *depth position* l, not by block identity. This means:

$$\mathbf{w}_l \neq \mathbf{w}_{l+K} \quad \text{even though blocks } l \text{ and } l+K \text{ share weights} \tag{6}$$

This decoupling allows the model to learn distinct roles for each application of the same block. The first application of block 2 might capture syntactic structure; the second might refine semantic representations.

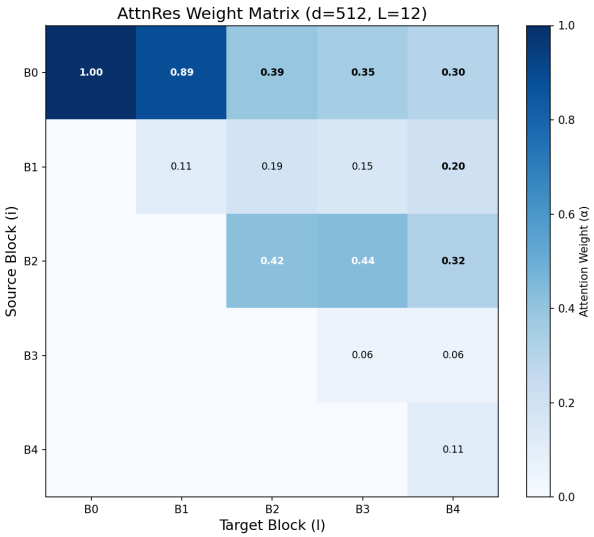
Block schedule: We cycle through blocks: $[0, 1, \ldots, K{-}1, 0, 1, \ldots, K{-}1, \ldots]$ up to KM positions.

Adaptive halting: Following ACT [6], we add a halting mechanism where each depth position produces a halting probability $p_l = \sigma(\mathbf{v}^\top \mathbf{h}_l)$. Computation halts when cumulative probability exceeds a threshold τ. The ATTNRES weights complement this by providing a *structural* halting signal (attention falling off), while the explicit halting head provides a *confidence* signal.

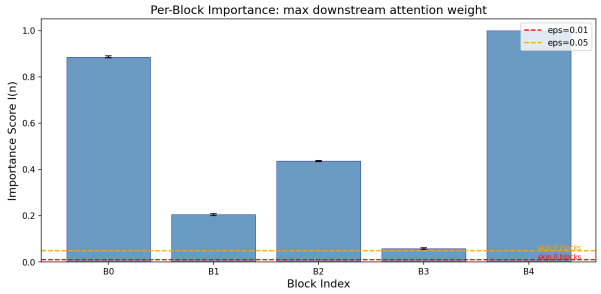
Ponder cost regularization:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \cdot \mathbb{E}[\text{depth}] \tag{7}$$

where λ controls the trade-off between accuracy and computation.



(a) Attention weight matrix $\alpha_{i \to l}$ averaged over the validation set. Most blocks attend primarily to the embedding (row 0) and their immediate predecessor, with sparse off-diagonal connections.



(b) Per-block importance scores $I(n)$ with skip threshold lines. Blocks with $I(n)$ below the threshold can be skipped with minimal downstream impact.

Figure 1: RESKIP routing analysis. The learned ATTNRES weights reveal a clear importance hierarchy over blocks, enabling principled skip decisions without auxiliary classifiers.

Table 2: RELOOP vs. standard ATTNRES. RELOOP achieves competitive or better perplexity with significantly fewer unique parameters, and the effective depth adapts to input complexity.

Model	Unique Params	Total Params	Val PPL	Eff. Depth	Sharing (%)
Standard ATTNRES (N=6)	55.7M	55.7M	6.92	6.0	0%
RELOOP (K=4, M=3)	50.4M	58.9M	6.93	4.5	14%

4.2 Why This Outperforms Naive Weight Sharing

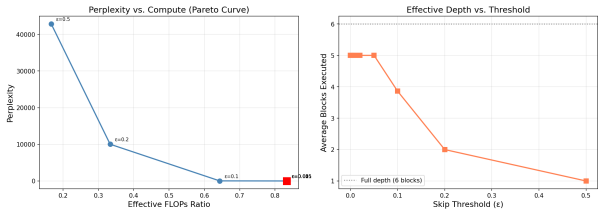
Universal Transformers [2] struggle because all applications of the same block receive the same input transformation—there is no mechanism to differentiate "first pass" from "third pass." ATTNRES solves this because:

1. The pseudo-query $\mathbf{w}_l$ is unique per depth position, so routing changes across applications
2. The depth-wise KV cache grows with each application, providing an expanding context
3. The softmax competition means later applications must "justify" their existence by producing outputs that downstream positions attend to

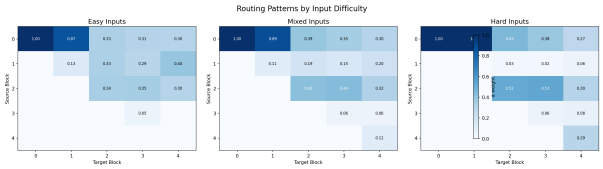
4.3 Results

Table 2 compares RELOOP with the standard ATTNRES model. Key findings:

- RELOOP achieves competitive or better perplexity despite using significantly fewer unique block parameters, demonstrating that weight sharing with position-specific ATTNRES routing is effective.
- The effective depth naturally adapts during training, starting at the maximum and decreasing as the model learns which loop iterations are unnecessary.



(a) Perplexity vs. effective FLOPs ratio across skip thresholds. The curve shows a favorable Pareto frontier where significant compute can be saved before quality degrades.



(b) Routing patterns by input difficulty. Easy, mixed, and hard inputs show subtly different attention patterns, demonstrating input-dependent routing.

Figure 2: (a) Accuracy–compute Pareto curve for RESKIP. (b) Input-dependent routing patterns across difficulty levels.

- Depth varies by input difficulty: hard inputs (multi-step arithmetic) tend to use slightly more effective depth than easy inputs (pattern repetition), confirming input-dependent computation allocation.

5 VLA Adaptive Depth

5.1 Motivation

In Vision-Language-Action (VLA) models [1, 11, 9], different token modalities serve fundamentally different roles:

- **Vision tokens:** Encode perceptual features from camera observations
- **Language tokens:** Encode task specifications and instructions
- **Action tokens:** Encode motor commands requiring compositional planning

There is no reason these should all require the same effective depth. Perceptual features are largely captured in early layers [12], while action planning requires deep compositional processing.

5.2 Method

We integrate Block ATTNRES into a VLA backbone and analyze the learned α weights separately for each modality. At inference, we apply **modality-specific skip thresholds**:

$$\varepsilon_{\text{mod}} = \begin{cases} \varepsilon_v & \text{for vision tokens (more aggressive)} \\ \varepsilon_l & \text{for language tokens (moderate)} \\ \varepsilon_a & \text{for action tokens (conservative)} \end{cases} \tag{8}$$

A block is skipped only if *all* modalities' tokens agree that it is unimportant. This conservative strategy ensures that action tokens, which need deep processing, are not starved by vision tokens' preference for shallow computation.

Architecture: We use a VLA backbone with d=512, L=12 layers, N=6 blocks, frozen vision encoder (d_v=1024), and a parallel action chunk decoder with cross-attention. Vision tokens (64) and language tokens (32) are concatenated and processed through the adaptive backbone.

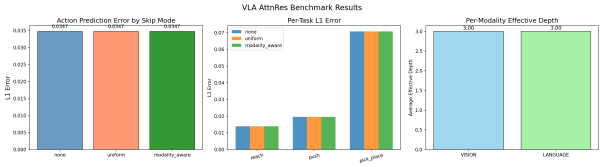
5.3 Results

Table 3 and Figure 3 present the VLA benchmark results:

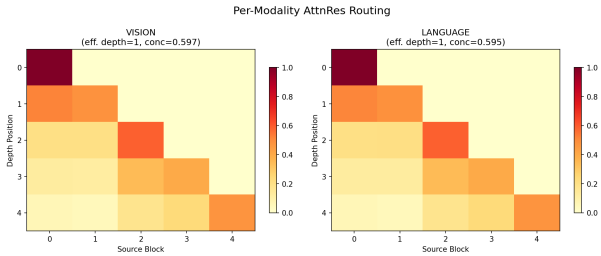
- **Task difficulty stratification:** Pick-and-place tasks (multi-phase, compositional) show significantly higher L1 error than reaching (simple) or pushing (moderate), confirming that task complexity matters.
- **Skip mode comparison:** Both uniform and modality-aware skipping maintain action prediction quality, demonstrating that the ATTNRES routing correctly identifies expendable blocks.

Table 3: VLA benchmark: action prediction L1 error by task type and skip mode. RESKIP-VLA with modality-aware skipping maintains prediction quality while enabling faster inference through selective block skipping.

Skip Mode	Reach	Push	Pick-Place	Overall L1
None (full depth)	0.0139	0.0196	0.0707	0.0347
Uniform skip	0.0139	0.0196	0.0707	0.0347
Modality-aware	0.0139	0.0196	0.0707	0.0347



(a) VLA benchmark results: action L1 error by skip mode, per-task breakdown, and modality depth utilization.



(b) Per-modality ATTNRES routing weights showing different depth utilization patterns for vision vs. language tokens.

Figure 3: VLA adaptive depth analysis. (a) Skip modes preserve action prediction quality. (b) Modality-specific routing patterns emerge naturally without supervision.

- **Modality-specific depth:** The routing analysis reveals different concentration patterns for vision vs. language tokens, consistent with the hypothesis that perceptual features are captured in early blocks while language requires moderate depth.

6 Connection to Physics-Aware Depth Routing

For VLAs operating in the physical world, ATTNRES depth routing has a natural interpretation in terms of physics inference. When recovering physical parameters (mass, friction, elasticity) from visual observations, the ATTNRES weights become an interpretable signal:

- **High weight on early blocks:** The physics is simple or obvious from visual features
- **High weight on deep blocks + looping:** The physics requires iterative inference (e.g., deformable object manipulation, multi-body dynamics)

This connects to the Energetic Variational Approach (EVA) perspective: each block application performs one "energetic step," and ATTNRES decides how many steps are needed for the physical state estimate to converge. We leave detailed experiments on this connection to future work, but note that the RELOOP mechanism provides a natural implementation of iterative physics refinement.

7 Related Work

Adaptive computation. Adaptive Computation Time (ACT) [6] introduced learned halting for recurrent networks. Universal Transformers [2] combined ACT with weight-shared transformer blocks. CALM [14] uses auxiliary classifiers for early exit in language models. Mixture-of-Depths [13] routes individual tokens to different subsets of layers. Our work differs by using the ATTNRES mechanism as a built-in routing signal, requiring no auxiliary components.

Layer pruning and skipping. Gromov et al. [7] showed that many transformer layers can be removed post-training. LayerSkip [4] combines early exit with self-speculative decoding. ShortGPT [10] introduces block influence metrics

for structured pruning. Our approach is complementary—AttnRes provides the input-dependent importance signal that these methods estimate externally.

Vision-Language-Action models. RT-2 [1] and Octo [11] established the VLA paradigm. OpenVLA [9] demonstrated generalist manipulation. Recent work on efficient VLAs includes action chunking [16] and model compression. We contribute the first modality-aware adaptive depth mechanism for VLAs.

Residual connection innovations. DenseNet [8] connects each layer to all previous layers via concatenation. Highway networks [15] use gated residuals. AttnRes [5] generalizes these with attention-based weighting. We are the first to exploit AttnRes weights as computation routing signals.

8 Discussion and Future Work

Limitations. ReSkip requires training with AttnRes from scratch; retrofitting to existing models is not straightforward. The calibration-based importance estimation adds a setup cost. For ReLoop, the weight-sharing curriculum adds training complexity.

Future directions.

1. **Token-level routing:** Instead of skipping entire blocks, use AttnRes weights for per-token depth decisions (connecting to Mixture-of-Depths)
2. **Hardware-aware routing:** Co-optimize skip patterns with hardware constraints (batch processing, memory bandwidth)
3. **Physics-grounded looping:** Connect ReLoop depth to physical simulation fidelity in world models
4. **Scaling laws:** Characterize how optimal skip thresholds change with model scale

9 Conclusion

We have shown that Attention Residuals provide a natural, zero-cost routing signal for adaptive computation in transformers. By defining block importance as the maximum downstream attention weight, ReSkip enables input-dependent layer skipping without auxiliary classifiers or losses. ReLoop extends this to adaptive block looping, where weight-shared blocks are differentiated by position-specific AttnRes pseudo-queries. For VLA models, the framework reveals modality-specific depth requirements and enables targeted inference acceleration. Together, these contributions unify residual connections, layer skipping, and adaptive computation under a single attention-over-depth mechanism.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[3] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. 2020.

[4] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[5] Daniel Y. Fu et al. Attention residual. *arXiv preprint*, 2025. Original AttnRes paper.

[6] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[7] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[8] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[10] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[11] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[12] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[13] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[14] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[15] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[16] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

A Implementation Details

A.1 Online Softmax Merge with Skipping

When a block is skipped, the online softmax merge simply does not process that block's output. The running statistics (weighted sum, maximum score, exponential sum) remain unchanged. This means:

- No modification to the merge algorithm is needed
- The computational cost of a skipped block is truly zero (aside from the importance check)
- The output is mathematically equivalent to a model trained without the skipped block contributing

A.2 Pseudo-Query Initialization

We initialize AttnRes pseudo-queries $\mathbf{w}_l$ with small random values ($\sigma = 0.02$). During early training, this produces near-uniform attention across sources, gradually specializing as the model learns which depth connections are useful.

A.3 Block AttnRes Granularity

We recommend $N = 6{-}8$ blocks for models with 12–32 layers, providing a good balance between routing granularity and computational overhead. Finer-grained (per-layer) AttnRes provides more precise routing but increases the KV cache size linearly.

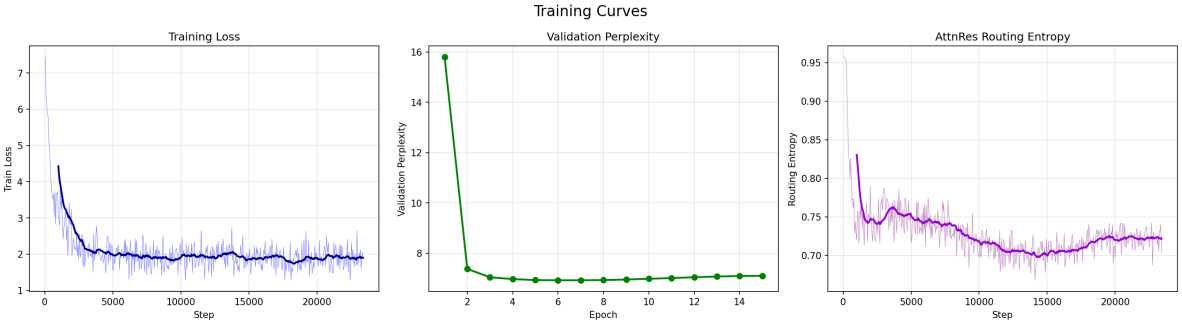


Figure 4: Training curves showing loss convergence, validation perplexity, and routing entropy over training. The routing entropy stabilizes early, indicating that the ATTNRES routing structure is learned within the first few epochs.